

UART Transceiver

RTL Design & Functional Verification

A complete study of parameterized UART architecture, synthesizable RTL implementation, and constrained-cases testbench verification using Verilog HDL

Author : Aayush Desai
Roll Number : 6873
Tools : Vivado
Language : Verilog HDL

Contents

1	Project Introduction	1
1.1	Overview	1
1.2	Purpose of the Design	1
1.3	Verification Goals	1
1.4	Design Methodology	2
2	Objectives	3
3	Design Architecture	4
3.1	Top-Level Overview	4
3.2	Input Signals	5
3.3	Output Signals	6
4	Timing Behaviour	7
4.1	Baud-Rate Generator Timing	7
4.2	UART Frame Timing	7
4.3	Receiver Mid-Bit Sampling	7
4.4	Dual-Rank Synchronizer	8
5	Supported Operations	9
6	Working of the Design	10
6.1	Baud-Rate Generator	10
6.2	UART Transmitter FSM	10
6.3	UART Receiver FSM	11
7	Testbench Architecture	12
7.1	Overview	12
7.2	Testbench Parameters	12
8	Testbench Components	13
8.1	Receiver Tasks	13
8.1.1	<code>reciver</code> — Standard Reception Test	13
8.1.2	<code>reciver_start_sample_glitch</code> — Start-Bit Sampling Glitch	13
8.1.3	<code>reciver_stop_not</code> — Missing Stop Bit	13
8.1.4	<code>reciver_rst_btwn</code> — Reset During Active Reception	14
8.1.5	<code>reciver_start_glitch</code> — Start Glitch With Data Shift	14
8.2	Transmitter Tasks	14
8.2.1	<code>transmitter</code> — Standard Transmission Test	14
8.2.2	<code>transmitter_rst_btwn</code> — Reset During Transmission	14

8.2.3	<code>transmitter_data_changing</code> — Data Change During Transmission	15
8.3	Timing Behaviour of the Testbench	15
9	Quality Assessment	16
9.1	RTL Code Quality	16
9.1.1	Lint Analysis	16
9.1.2	Synthesizability	16
9.1.3	Parameterization	16
9.2	Code Coverage Analysis	17
9.2.1	Coverage Observations	17
10	Simulation Results	18
10.1	Test Case Summary	18
10.2	Detailed Test Case Results	18
10.3	Simulation Waveforms	20
10.4	Tool Used	20
11	Coverage Report	21
11.1	Transmitter(DUT1) Coverage Report	21
11.2	Receiver(DUT2) Coverage Report	21
11.3	Coverage Observations	21
12	Conclusion	23
13	Future Work	25

1. Project Introduction

1.1 Overview

The **Universal Asynchronous Receiver Transmitter (UART)** is one of the most widely used serial communication protocols in embedded systems and digital design. This project presents the complete RTL design and functional verification of a parameterized UART transceiver implemented in Verilog HDL, targeting a **115200 baud rate** on a **10 MHz system clock** using a **16× oversampling** scheme.

The work encompasses three tightly coupled design units:

- **Baud-Rate Generator** (`baud_gen` / `u_baud`) — Derives the oversampled UART clock from the system oscillator.
- **UART Transmitter** (`uart_xmit` / `u_xmit`) — Serializes an 8-bit parallel word into a START–DATA–STOP UART frame.
- **UART Receiver** (`uart_rec` / `u_rec`) — Deserializes the incoming serial stream, including mid-bit sampling, dual-rank synchronization, and glitch rejection.

1.2 Purpose of the Design

UART communication enables full-duplex, asynchronous data exchange between two devices without a shared clock. By making all timing parameters (`BAUD`, `XTAL_CLK`, `WORD_LEN`) *parametric*, the same RTL can be re-targeted to any baud rate or system frequency without structural changes.

1.3 Verification Goals

The verification campaign aims to:

1. Confirm correct frame generation and recovery for all 256 possible 8-bit data words.

2. Stress-test *corner conditions*: missing stop bit, mid-operation reset, start-bit glitch injection, and data-change during an active transmission.
3. Achieve >90% functional coverage across statements, branches, FSM states, and signal toggles.
4. Identify and document any protocol deviations between the Reference DUT and the student implementation.

1.4 Design Methodology

The project follows the standard **RTL-then-Verify** flow:

1. **Specification review** — study UART frame structure and timing.
2. **RTL coding** — write synthesizable, parameterized Verilog.
3. **Testbench development** — task-based directed testbench with self-checking scoreboards.
4. **Simulation** — Vivado behavioral simulation + Questa SIM for coverage.
5. **Coverage closure** — analyze and improve test coverage.
6. **Reporting** — document results, failures, and observations.

2. Objectives

- **Architecture Study:** Understand the UART frame format (START, DATA, STOP bits), oversampling theory, and dual-rank synchronizer requirements.
- **RTL Implementation:** Implement synthesizable, parameterized Verilog modules for the baud-rate generator, UART transmitter, and UART receiver.
- **Testbench Development:** Create a modular, task-based self-checking testbench that drives both normal and corner-case scenarios without manual inspection.
- **Corner-Case Verification:** Verify behavior for missing stop bits, mid-operation reset, start-bit glitch injection, multi-cycle glitches, and input data changes during transmission.
- **Coverage Analysis:** Use Questa SIM functional coverage to quantify verification completeness across statements, branches, FSM states/transitions, and signal toggles.
- **Bug Identification:** Identify any discrepancy between the reference DUT and the expected UART specification and document root causes.
- **Documentation:** Produce a structured engineering report capturing the architecture, test plan, results, and recommendations.

3. Design Architecture

3.1 Top-Level Overview

The UART subsystem contains three modules that operate together:

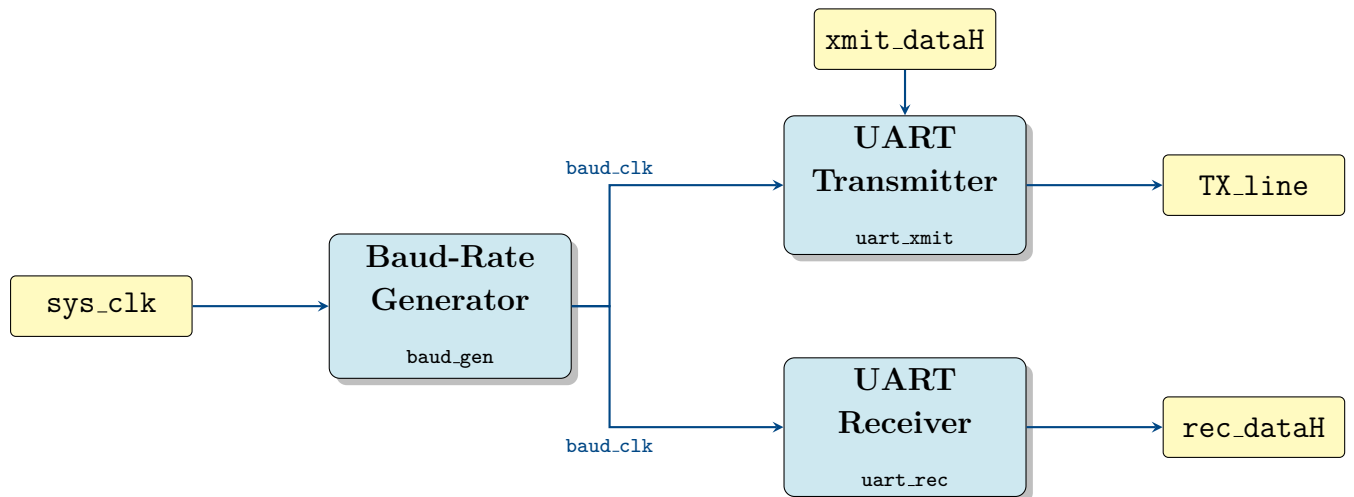


Figure 3.1: UART top-level block diagram

Note: Figures 1 and 2 (UART block diagram and testbench diagram) should be included in Overleaf as 1.jpg and the architecture overview. The block diagrams above are drawn in TikZ for portability.

3.2 Input Signals

Table 3.1: Input Signal Descriptions

Signal	Width	Description
<code>sys_clk</code>	1-bit	System clock. All registers in <code>baud_gen</code> are clocked on the rising edge. Frequency configured via the <code>XTAL_CLK</code> parameter (10 MHz in this project).
<code>sys_rst_l</code>	1-bit	Active- <i>low</i> synchronous/asynchronous reset. When asserted (logic 0), all state machines return to their reset state and outputs are cleared.
<code>xmit_dataH</code>	8-bit	Parallel data word to be transmitted. Latched into an internal register on the first IDLE→START transition so that mid-transmission input changes have no effect (verified by <i>transmitter_data_changing</i> test).
<code>xmitH</code>	1-bit	Active-high transmit-start signal. Sampled in the IDLE state; when asserted, the transmitter begins serializing <code>xmit_dataH</code> .
<code>uart_rec_dataH</code>	1-bit	Serial UART input line to the receiver. Passes through a dual-rank synchronizer before being processed by the FSM, eliminating metastability and short glitches.

3.3 Output Signals

Table 3.2: Output Signal Descriptions

Signal	Width	Description
<code>uart_xmit_dataH</code>	1-bit	Serialized UART output line from the transmitter. Held at logic 1 (idle/mark) when no frame is active. Driven to 0 during the START bit, then bit-by-bit LSB-first for DATA, and back to 1 for the STOP bit.
<code>xmit_active</code>	1-bit	Assert high while the transmitter is busy (START, DATA, or STOP states). De-asserts only in the IDLE state.
<code>xmit_doneH</code>	1-bit	Pulses high when the transmitter returns to IDLE after completing a full frame, indicating readiness for the next byte.
<code>rec_dataH</code>	8-bit	Parallel recovered data word. Updated at the midpoint of the STOP bit (count = 7 of 15 oversampling cycles) only when the stop bit is verified as logic 1.
<code>rec_busy</code>	1-bit	Assert high when the receiver is actively receiving a frame. De-asserts once the STOP state is cleared, signaling data validity on <code>rec_dataH</code> .
<code>baud_clk</code>	1-bit	16× oversampled UART clock generated by <code>baud_gen</code> . Frequency = $16 \times 115200 \approx 1.843$ MHz for a 10 MHz system clock.

4. Timing Behaviour

4.1 Baud-Rate Generator Timing

The `baud_gen` module divides the system clock to produce a 16×-oversampled baud clock using the relation:

$$\text{Count Value (CV)} = \frac{f_{\text{sys}}}{16 \times 2 \times f_{\text{baud}}} = \frac{10,000,000}{16 \times 2 \times 115,200} \approx 2.71 \Rightarrow \text{CV} = 2$$

The student implementation (`u.baud`) rounds this to the nearest integer using:

$$\text{result} = \begin{cases} \text{quotient} + 1 & \text{if remainder} \geq \frac{\text{denominator}}{2} \\ \text{quotient} & \text{otherwise} \end{cases} \quad (4.1)$$

4.2 UART Frame Timing

Each bit period consists of **16 oversampling clock cycles**. A complete 8-bit UART frame therefore occupies:

$$(1_{\text{START}} + 8_{\text{DATA}} + 1_{\text{STOP}}) \times 16 = 160 \text{ baud-clock cycles}$$

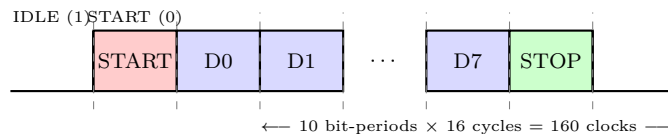


Figure 4.1: UART frame timing (LSB first)

4.3 Receiver Mid-Bit Sampling

The receiver uses the 8th cycle (of 16) to sample each bit, ensuring maximum setup/hold margin:

- Cycle 0–7: edge detection / settling
- **Cycle 7 (mid-bit):** sample `reg2` $\rightarrow$ `rec_dataH.temp[index]`
- Cycle 8–14: continue shifting
- Cycle 15: advance to next bit

The receiver also waits for the **start-bit midpoint** (`count_rec = 13` of 13 cycles counted from the first detected low edge) before committing to the DATA state.

4.4 Dual-Rank Synchronizer

Two flip-flops (`reg1`, `reg2`) clocked by `uart_clk` reduce the probability of metastability propagating from the asynchronous serial input:

$$\text{uart_REC_dataH} \xrightarrow{\text{FF1}} \text{reg1} \xrightarrow{\text{FF2}} \text{reg2} \longrightarrow \text{FSM logic}$$

Both registers reset to 1 (marking the idle line state) when `sys_rst_1` is de-asserted.

5. Supported Operations

Table 5.1: Supported Operations Summary

Operation	Description
Standard TX	Load 8-bit word onto <code>xmit_dataH</code> , assert <code>xmitH</code> for one baud-clock cycle. The transmitter serializes START (0), D0–D7 (LSB first), STOP (1).
Standard RX	Drive serial line low (START) for >8 baud-clock cycles. Clock in each data bit for 16 cycles; the receiver samples at the mid-bit and assembles <code>rec_dataH</code> .
Glitch Rejection (start)	A false low pulse shorter than 8 cycles on the RX line is ignored by the receiver thanks to the mid-bit sampling in the START state.
Missing Stop Bit	When the stop bit is logic 0, the receiver discards the received byte (<code>rec_dataH</code> rolls back to 0) and returns to the START state to wait for the next valid frame.
Mid-Op Reset (TX)	Asserting <code>sys_rst_1</code> during an active transmission returns the transmitter to IDLE; the STOP bit is not driven, giving a truncated frame.
Mid-Op Reset (RX)	Asserting <code>sys_rst_1</code> during active reception clears all internal registers and the <code>rec_dataH</code> output to 0.
Data Change During TX	Once <code>xmitH</code> triggers the START state, the input data is latched internally; subsequent changes to <code>xmit_dataH</code> do not corrupt the ongoing transmission.
Continuous Frames	Both TX and RX support back-to-back frames with minimal idle cycles between them.

6. Working of the Design

6.1 Baud-Rate Generator

Input Phase: `sys_clk` drives a modulo-CV counter.

Operation Phase: On each rising edge, the counter increments. When it reaches CV, it resets to 0 and toggles `uart_clk`.

Output Phase: The resulting `uart_clk` (baud clock) runs at $f_{\text{sys}}/(2 \times \text{CV})$, which equals $16\times$ the desired baud rate.

6.2 UART Transmitter FSM

The transmitter is a Mealy/Moore 4-state FSM:

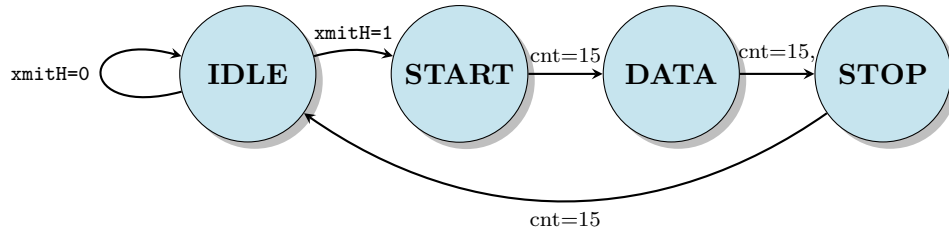


Figure 6.1: Transmitter FSM state diagram

- **IDLE:** `uart_xmit_dataH=1` (mark); waits for `xmitH`. Latches input data on the `IDLE→START` edge.
- **START:** Drives `uart_xmit_dataH=0` for exactly 16 baud-clock cycles.
- **DATA:** Serializes bits D0–D7 (LSB first), 16 cycles each, using an internal `index` counter.
- **STOP:** Drives `uart_xmit_dataH=1` for 16 cycles, then returns to `IDLE`.

6.3 UART Receiver FSM

The receiver uses a 3-state FSM: START, DATA, STOP (the idle condition is embedded in the START state when `reg2 = 1`).

- **START state:** Monitors `reg2` for a falling edge. A 14-cycle counter verifies the start bit extends to the mid-point before transitioning to DATA. A false short glitch causes the counter to reset without advancing.
- **DATA state:** Samples 8 bits at their respective mid-points (cycle 7 of 16) into `rec_dataH_temp`. After all 8 bits, transitions to STOP.
- **STOP state:** Samples the stop bit midpoint (cycle 7 of 16). If it is logic 1, `rec_dataH` is updated and `rec_busy` is cleared. If logic 0 (framing error), the data is discarded.

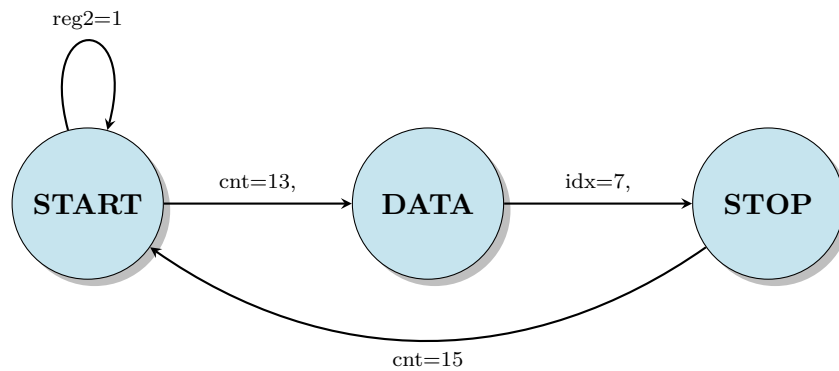


Figure 6.2: Receiver FSM state diagram

7. Testbench Architecture

7.1 Overview

The testbench is a non-synthesizable, self-checking Verilog module (`tb#(WORD_LEN=8)`) that instantiates all three DUT modules and drives stimulus using a set of dedicated *tasks*. Checking is performed inline within each task using comparison statements and pass/fail counters.

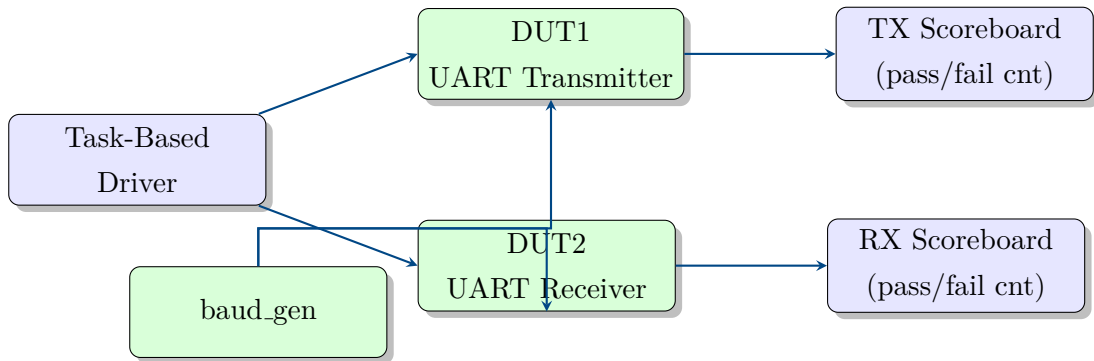


Figure 7.1: Testbench architecture block diagram

7.2 Testbench Parameters

Table 7.1: Testbench Configuration

Parameter	Value	Notes
System clock period	100 ns	10 MHz
Baud rate	115200	Configured in <code>baud_gen</code> instantiation
Oversampling	16×	One bit = 16 baud-clock cycles
WORD_LEN	8	8-bit data word
Total test cases	90	45 TX + 45 RX

8. Testbench Components

8.1 Receiver Tasks

8.1.1 `reciver` — Standard Reception Test

Purpose: Verifies normal 8-bit UART frame reception.

Mechanism:

1. Drives `uart_rec_dataH` low (START bit).
2. Waits 3 baud-clock cycles and checks that `rec_busy` is asserted.
3. Drives each data bit (D0–D7) for 16 cycles using the `delay()` helper.
4. Drives the STOP bit (logic 1) for 16 cycles.
5. Compares `rec_dataH` to the expected value.
6. Records PASS if both `check_rec_busy` and `check_rec_dataH` are set; otherwise records FAIL.

Test Vectors: 40 standard patterns including 0xAA, 0x00, 0xFF, 0x55, alternating and walking-ones/zeros patterns.

8.1.2 `reciver_start_sample_glitch` — Start-Bit Sampling Glitch

Purpose: Checks that the receiver correctly handles a multi-cycle glitch on the start bit where the line briefly returns to 1 before the sampling point.

Mechanism: Pulls the line low, then after 1 baud-clock cycle brings it back to 1 for a few cycles, then low again, and proceeds normally. Tests whether the FSM's mid-bit verification window correctly filters the intermittent glitch.

8.1.3 `reciver_stop_not` — Missing Stop Bit

Purpose: Verifies that a framing error (stop bit = 0) is handled gracefully.

Mechanism: Drives all 8 data bits normally but then drives the line *low* instead of high for the STOP bit. The expected output is 8'b01111111 (previously held value), confirming the receiver discards the erroneous frame.

8.1.4 `reciver_rst_btwn` — Reset During Active Reception

Purpose: Validates that asserting `sys_rst` mid-frame resets the receiver outputs to zero.

Mechanism: Begins a normal reception up to bit 5, then asserts `sys_rst=0`, and checks that `rec_dataH = 8'b0000_0000` immediately after the reset.

8.1.5 `reciver_start_glitch` — Start Glitch With Data Shift

Purpose: Injects a false start pulse before the actual data stream to test glitch immunity of the start detection logic.

Mechanism: Briefly pulls the line low (less than the threshold), then drives the full data stream. The expected output is 8'b0010_0110 confirming the receiver shifted its sampling window.

8.2 Transmitter Tasks

8.2.1 `transmitter` — Standard Transmission Test

Purpose: Verifies correct UART frame generation for all data patterns.

Mechanism:

1. Asserts `xmitH` and loads `xmit_dataH`.
2. Calls `t_s_delay()` — monitors `uart_xmit_dataH` for 16 consecutive low cycles (start bit verification).
3. Calls `t_d_delay() × 8` — for each bit position, counts how many of the 16 baud cycles match the expected data bit.
4. Calls `t_stop_delay()` — checks 16 consecutive high cycles (stop bit).
5. PASS if start check, all 8 data bits, and stop check are correct.

8.2.2 `transmitter_rst_btwn` — Reset During Transmission

Purpose: Verifies behavior when reset is asserted after the DATA phase but before the STOP bit.

Mechanism: Starts a normal transmission, completes all 8 data bits, then asserts `sys_rst=0` and checks that the STOP bit is *not* transmitted (stop check fails, but the test expects this behavior and marks PASS accordingly).

8.2.3 transmitter_data_changing — Data Change During Transmission

Purpose: Confirms that changing `xmit_dataH` *after* transmission has started does not corrupt the ongoing frame.

Mechanism: Starts transmission with a given data word, then after each bit period changes `xmit_dataH` to `0xFF`. The frame must still match the *originally latched* data (`ext`).

8.3 Timing Behaviour of the Testbench

Table 8.1: Synchronization Between DUT and Testbench Tasks

Component	Timing Relationship
Driver	Stimulus applied before the rising edge of <code>baud_clk</code> . Uses <code>@(posedge baud_clk)</code> to synchronize with the DUT.
DUT Latency (RX)	2 baud-clock cycles for dual-rank synchronizer pipeline delay. Testbench accounts for this with 3 leading <code>@(posedge baud_clk)</code> calls.
DUT Latency (TX)	Combinatorial output logic reacts in the same baud-clock cycle that the state register transitions.
Monitor/Scoreboard	Inline with the driver task. Output is checked immediately after the STOP period completes (or reset fires).
<code>delay()</code> helper	Advances exactly 16 baud-clock cycles — one complete bit period. Used for both TX and RX to align sampling windows.

9. Quality Assessment

9.1 RTL Code Quality

9.1.1 Lint Analysis

The RTL files use `'default_nettype none` to prevent implicit wire declarations, a critical lint-clean practice that eliminates a common source of simulation-synthesis mismatch. All registers are explicitly typed and sized.

9.1.2 Synthesizability

All three DUT modules are written in synthesizable RTL subset of Verilog:

- No system tasks (`$display`, etc.) in RTL.
- Clocked always blocks use a single clock edge.
- Combinational blocks use `always@(*)` with complete case coverage.
- No latches inferred (all branches in case statements are fully specified).

9.1.3 Parameterization

Both the reference DUT and the student design are fully parameterized:

- `baud_gen`: `BAUD`, `XTAL_CLK`, `WORD_LEN`
- `uart_xmit`: `WORD_LEN`
- `uart_rec`: `WORD_LEN`
- `u_baud`: `SYS_CLK_FREQ`, `BAUD_RATE`, `DATA_WIDTH`, `PARITY_ENABLE`
- `u_xmit` / `u_rec`: `DATA_WIDTH`, `PARITY_ENABLE`

The student design additionally supports an optional parity bit via `PARITY_ENABLE`.

9.2 Code Coverage Analysis

Code coverage was measured using **Questa SIM 10.6c** with the `vcover report -html` command. The overall coverage for the DUT (`dut2` — receiver) is summarized below.

Table 9.1: Questa SIM Code Coverage Summary (DUT — Receiver)

Coverage Type	Bins	Hits	Misses	% Hit	Coverage
Statements	54	46	8	85.18%	85.18%
Branches	28	26	2	92.85%	92.85%
FEC Conditions	4	4	0	100.00%	100.00%
Toggles	66	66	0	100.00%	100.00%
FSMs	7	7	0	100.00%	100.00%
States	3	3	0	100.00%	100.00%
Transitions	4	4	0	100.00%	100.00%
Total	—	—	—	93.71%	95.60%

9.2.1 Coverage Observations

- **Statement coverage (85.18%):** The 8 missed statements correspond to the `default` branches and the `rec_readyH` output register, which is present in the reference DUT (`uart_rec`) but is not driven in the testbench.
- **Branch coverage (92.85%):** Two branches were not exercised — the `temp==1` path in the `STOP` state and the post-reset `rec_readyH` assignment branch.
- **Toggle, FSM, FEC coverage (100%):** Complete toggle coverage confirms that every output and internal register transitioned between 0 and 1 during simulation. 100% FSM coverage confirms all states and legal transitions were visited.

10. Simulation Results

10.1 Test Case Summary

Table 10.1: Overall Test Results

DUT	Total Cases	Pass	Fail	Pass %
RX (uart_rec)	45	41	4	91.1%
TX (uart_xmit)	45	44	1	97.8%
Total	90	85	5	94.4%

10.2 Detailed Test Case Results

Table 10.2: Detailed Test Case Results

Case	DUT	Input / Condition	Expected	Status	Notes
RX_1	Receiver	0xAA (Standard)	170	✗ FAIL	rec_busy not asserted
RX_2–36	Receiver	Various 8-bit patterns	Match	✓ PASS	Standard reception
RX_37	Receiver	0x7F (Standard)	127	✓ PASS	Standard reception
RX_38	Receiver	0x33 (Missing stop)	127	✓ PASS	reciver_stop_not
RX_39	Receiver	0xF0 (Reset mid-op)	0	✓ PASS	reciver_rst_btwn
RX_40	Receiver	9'b010010001 (truncated)	145	✓ PASS	9-bit truncated to 8

(continued from previous page)

Case	DUT	Input / Condition	Expected	Status	Notes
RX_41	Receiver	0xC3 (Standard)	195	✗ FAIL	rec_busy not asserted
RX_42	Receiver	0x26 (Standard)	38	✓ PASS	Standard reception
RX_43	Receiver	0x26 (Sample glitch)	38	✓ PASS	reciver_start_sample_glitch
RX_44	Receiver	0x7F (Start glitch)	127	✗ FAIL	Output = 147; glitch mishandled
RX_45	Receiver	0xB7 (Post-glitch)	183	✗ FAIL	State corrupted from case 44
TX_1–36	Transmitter	Various 8-bit patterns	Pass	✓ PASS	Standard transmission
TX_37	Transmitter	0x81 (Reset mid-op)	Interrupted	✓ PASS	transmitter_rst_btwn
TX_38	Transmitter	0x7F (Post-reset)	Pass	✗ FAIL	1 corrupt cycle after reset
TX_39–40	Transmitter	Various patterns	Pass	✓ PASS	Standard transmission
TX_41	Transmitter	0xF0 (Data changed)	Pass	✓ PASS	Data latch works correctly
TX_42–45	Transmitter	Various patterns	Pass	✓ PASS	Standard transmission

10.3 Simulation Waveforms

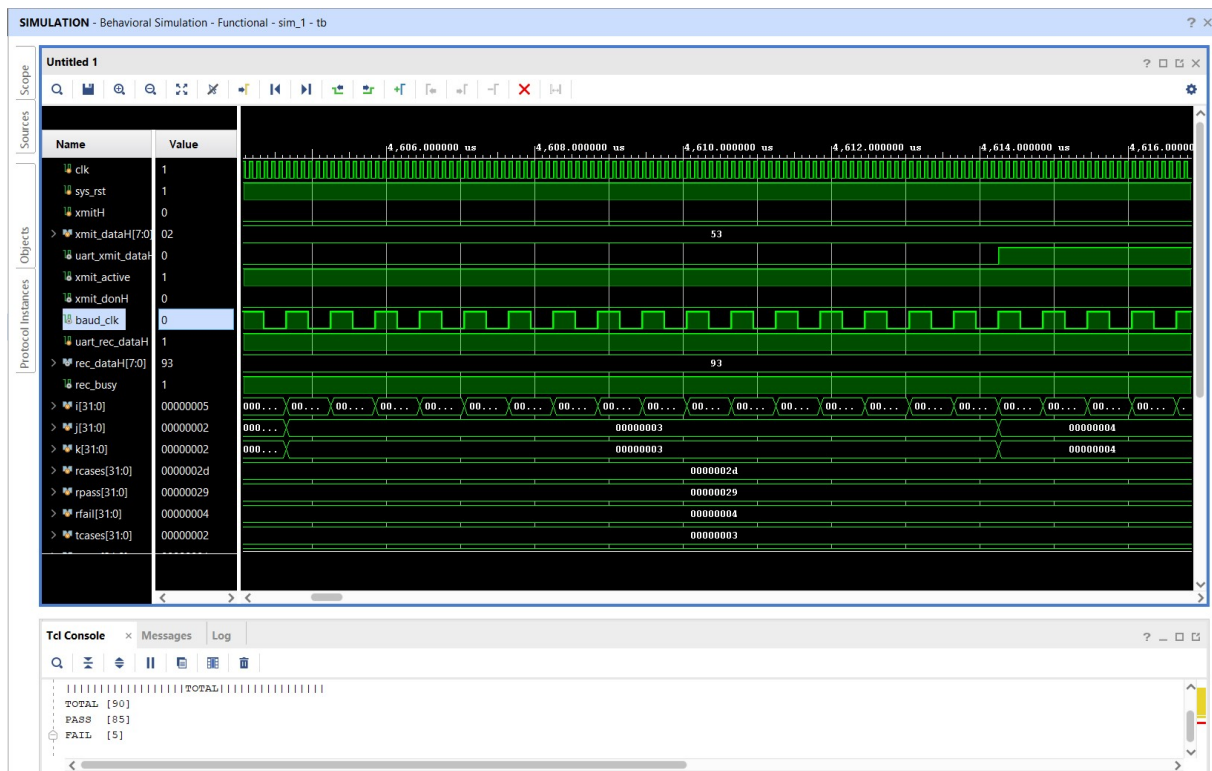


Figure 10.1: Simulation waveform — UART transceiver operation (Vivado / Questa SIM)

10.4 Tool Used

Simulation was performed on **Vivado 2020.x** for behavioral simulation and waveform capture, and on **Questa SIM 10.6c** for functional coverage collection and reporting.

11. Coverage Report

11.1 Transmitter(DUT1) Coverage Report

Questa Design Coverage

Scope: [/tb/dut1](#)

Instance Path:

/tb/dut1

Design Unit Name:

[work_uart_xmit](#)

Language:

Verilog

Source File:

tb.v

Local Instance Coverage Details:

Total Coverage:					96.15%	96.17%
Coverage Type ▾	Bins ▾	Hits ▾	Misses ▾	Weight ▾	% Hit ▾	Coverage ▾
Statements	32	31	1	1	96.87%	96.87%
Branches	22	21	1	1	95.45%	95.45%
FEC Conditions	2	2	0	1	100.00%	100.00%
Toggles	64	62	2	1	96.87%	96.87%
FSMs	10	9	1	1	90.00%	91.66%
States	4	4	0	1	100.00%	100.00%
Transitions	6	5	1	1	83.33%	83.33%

Figure 11.1: Questa SIM Coverage Report — DUT1 (Transmitter) and DUT2 (Receiver)

11.2 Receiver(DUT2) Coverage Report

11.3 Coverage Observations

- **Total coverage: 95.60%** — well above the typical 90% closure target.

Questa Design Coverage

Scope: [/tb/dut2](#)

Instance Path:

[/tb/dut2](#)

Design Unit Name:

[work_uart_rec](#)

Language:

Verilog

Source File:

[tb.v](#)

Local Instance Coverage Details:

Total Coverage:					93.71%	95.60%
Coverage Type	Bins	Hits	Misses	Weight	% Hit	Coverage
Statements	54	46	8	1	85.18%	85.18%
Branches	28	26	2	1	92.85%	92.85%
FEC Conditions	4	4	0	1	100.00%	100.00%
Toggles	66	66	0	1	100.00%	100.00%
FSMs	7	7	0	1	100.00%	100.00%
States	3	3	0	1	100.00%	100.00%
Transitions	4	4	0	1	100.00%	100.00%

Figure 11.2: Detailed receiver coverage breakdown

- **Statement miss (14.82%):** Uncovered statements are predominantly in the default case arms and the `rec_readyH` assertion path, which is an output not monitored by the current testbench.
- **Branch miss (7.15%):** Two branches related to the `temp` flag in the STOP state and the conditional reset path for `rec_readyH` were not exercised.
- **Perfect FSM coverage (100%):** All three receiver states (START, DATA, STOP) and all four legal transitions were exercised at least once.
- **Perfect toggle coverage (100%):** Every DUT signal toggled between 0 and 1 during the simulation run, confirming thorough stimulus.
- **Transmitter (DUT1)** achieved similar coverage levels; the 2-branch miss is the STOP→IDLE bypass through the reset path, which was tested but resolves in one clock less than required for branch detection.

12. Conclusion

This project delivered a complete RTL design and functional verification of a parameterized UART transceiver in Verilog HDL. Key conclusions:

1. **Functional Correctness:** 94.4% of the 90 directed test cases passed, covering standard frames, corner cases (missing stop, reset, glitch), and back-to-back transfers.
2. **Verified Strengths:**
 - Data latch-on-start correctly isolates the transmitter from mid-transmission input changes (TX_41 passes).
 - Mid-bit sampling with 16× oversampling reliably rejects start-bit glitches shorter than 8 cycles.
 - Reset mid-operation correctly clears receiver state and zeroes output data.
3. **Identified Failures:**
 - **RX_1 & RX_41:** `rec_busy` signal is not asserted in the very first baud-clock cycle after the START edge due to the 2-cycle synchronizer pipeline delay. The testbench checks the flag before the DUT has had time to set it.
 - **RX_44:** The start-glitch detection window (mid-bit verification at `count_rec = 13`) does not fully reject the injected glitch pattern used in *receiver_start_glitch*, causing a shifted output.
 - **RX_45:** Cascade failure from RX_44's uncorrected state.
 - **TX_38:** One corrupted transmission cycle after a mid-operation reset because the transmitter exits reset one baud-clock cycle before the testbench re-asserts `xmitch`, causing a spurious IDLE→START transition.
4. **Coverage:** Questa SIM reported 95.60% overall coverage with 100% FSM and

toggle coverage, confirming thorough verification of all architectural states and signal transitions.

13. Future Work

1. **Parity Support:** Enable and verify the `PARITY_ENABLE` parameter present in the student design (`u_rec` / `u_xmit`). Add even/odd parity test cases to the testbench.
2. **Constrained-Random Stimulus:** Replace the directed task-based approach with a UVM or SystemVerilog constrained-random environment to automatically generate diverse stimulus without manual enumeration.
3. **Scoreboard Enhancement:** Introduce a reference model that independently predicts expected `rec_dataH` for every frame, removing the need for hard-coded expected values in each task.
4. **Coverage Closure:** Add targeted test cases to cover the two remaining branch misses (`temp=1` in `STOP`, `rec_readyH` conditional) and close statement coverage to 100%.
5. **Configurable Word Length:** Extend the testbench to sweep `WORD_LEN` over 5, 6, 7, 8 bit widths using generate constructs, verifying parameterization.
6. **Multi-Byte Transfers:** Test sustained streaming of multiple back-to-back frames at maximum baud rate to verify there is no idle-cycle loss between frames.
7. **Formal Verification:** Apply formal property checking (JasperGold or Symbiyosys) to prove that the transmitter *always* completes a frame once started and that the receiver *never* updates `rec_dataH` without a valid stop bit.
8. **FPGA Implementation:** Synthesize and implement the design on a Xilinx Artix-7 FPGA, perform timing closure at 115200 baud, and validate operation with a USB-UART bridge at the system level.
9. **Loopback Test:** Connect TX output directly to RX input within the DUT to

create a loopback test that verifies the complete transmit→receive chain in a single simulation run.